PRACTICAL LAB REPORT

Local AI with Ollama: Running and Developing Local AI Applications

1. EXPERIMENTAL ENVIRONMENT

The lab uses Ollama as a local inference server (localhost:11434) accessed from Python through its REST API. Architecture:
Python → HTTP POST → localhost:11434 → Ollama → Local LLM → response → Pythonoping Local AI Applications

Operating system	Windows 11
Python version	Python 3.12.2
CPU / RAM	CPU:AMD Ryzen 7 RAM:16 GB
GPU / VRAM	GPU : NVIDIA GeForce RTX VRAM:8GB
Ollama version	0.40.11
Models used	Qwen3.5 (Model 1), Gemma3 (Model 2)

2. USING A FIRST LOCAL MODEL (EXERCISE 4)

Questions:

(1) Did the model produce different answers? Why? (2) Is the model communicating with a cloud API? (3) Where is inference being performed? (4) What happens if the Internet connection is disabled?

Answers

1. Yes. The model changes its language and level of detail depending on the audience. It can also give different answers to the same prompt because it generates text using probabilities.
2. No. The code uses localhost:11434, which means Ollama runs on my own computer. No API key or cloud server is needed.
3. On my computer. Ollama loads the model into memory and uses the GPU if available; otherwise, the CPU
4. It still works because the model is already downloaded and runs locally. The internet is only needed to download models

3. PROMPT ENGINEERING (EXERCISE 5)

Questions

Compare the two answers. Explain the role of: role · context · constraints · output format · specificity.

Answers

Yes, the answers are different because the prompts have different instructions.

- Role: Defines the model's voice and expertise.
- Context: Helps the model understand the audience and topic.
- Constraints: Control the length and complexity.
- Output format: Organizes the answer as requested.
- Specificity: Clear prompts give more precise answers, while vague prompts give more general answers.

4. MODEL MANAGEMENT (EXERCISE 6)

Questions:

What information do you get from ollama list vs ollama ps? What is the difference between a downloaded and a loaded model?

Answers

- ollama list shows the models stored on disk (name, ID, size, modification date). ollama ps shows the models currently in memory (size, CPU/GPU split, context length, time before unloading).
- A downloaded model is only a set of weight files that uses disk space. A loaded model has its weights read into RAM / VRAM and is ready to answer; it uses memory, and the first request after a cold start pays a loading delay. Ollama unloads an idle model after the keep_alive period (5 minutes by default).

5. COMPARING TWO LOCAL MODELS (EXERCISE 7)

Questions

Which model is faster? Which produces better code? Which uses more memory? Which would you choose for programming assistance? Is the largest model necessarily the best?

Answers

- Which model is faster?

Qwen3.5 generates text faster, but Gemma3 had a shorter total response time.

- Which produces better code?

Both scored 4/5 for code correctness. Qwen3.5 had a higher overall quality score.

- Which uses more memory?

Qwen3.5 uses more model memory.

- Which would you choose for programming assistance?

I would choose Qwen3.5 because it had a higher overall quality score.

- Is the largest model necessarily the best?

No. The best model depends on quality, speed, and memory usage.

[21]:

	Model	Response time (s)	Tokens/s	Function found	Correct on -5..1999	type hints	docstring	complexity
0	qwen3.5	24.3	37.0	True	False	True	True	True
1	gemma3	36.5	70.2	True	True	True	True	True

6. CONVERSATION MEMORY

Questions:

Why does the model remember the name? Is the LLM itself remembering?

Answers

- The model remembers the name because the application sends the previous conversation with each request. The LLM is stateless, so it does not store memories itself. The conversation history is stored in a Python list and sent through the /api/chat endpoint. As the conversation grows, the prompt gets longer and may exceed the context window, so old messages may need to be shortened or summarized.

7. STRUCTURED OUTPUT (EXERCISE 11)

Questions

Why is structured output important when integrating an LLM into a software application?

Answers

- Structured output is important because software needs predictable data. JSON Schema ensures the LLM returns data in the correct format, with the required fields and types. This makes it easier to validate and use the output in applications. However, correct formatting does not guarantee correct information.

8. LOCAL DOCUMENT ASSISTANT (EXERCISE 12)

Questions:

Does your model respect the "Information not available." instruction every time?

Answers

- In my test, the model answered the reinforcement learning question using the document. For "What is quantum computing?", it returned "Information not available."

This shows that the model can follow the instruction, but not always. Using a clear prompt can reduce hallucinations, but cannot eliminate them.

9. LOCAL VISION AI (OPTIONAL)

Questions

Difference between an LLM and a multimodal model? What additional information does the vision model receive? Which applications could use local vision AI?

Answers

An LLM processes text, while a multimodal model can process both text and images. It uses a vision encoder to understand images sent with the prompt. Local vision AI is useful for OCR, document analysis, and industrial inspection. It also helps protect privacy because images stay on the local machine.

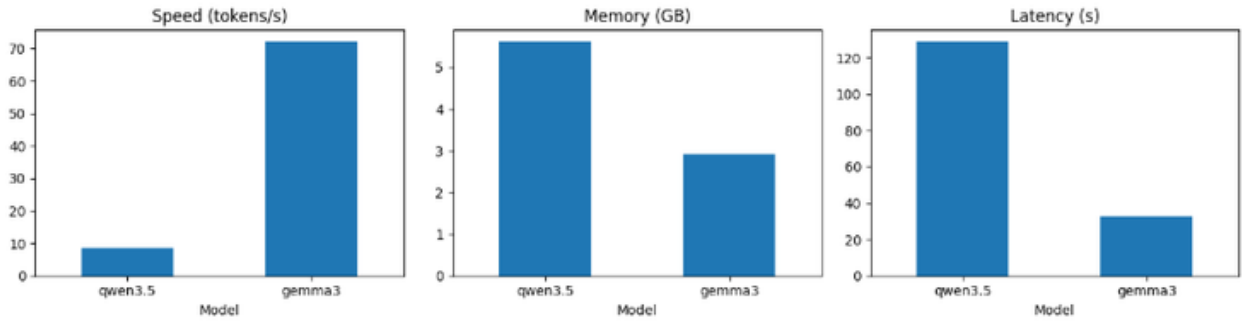
10. PERFORMANCE BENCHMARK

Discussion: the model-size trade-off.

Qwen3.5 is larger (6.55 GB) and uses more memory (5.61 GB), but it generates 38.03 tokens/s. Gemma3 is smaller (3.35 GB), uses 2.91 GB of memory, and generates 20.63 tokens/s. Gemma3 loads more slowly (19.08 s vs 0.01 s), but its response time is shorter (58.82 s vs 141.86 s).
Therefore, Qwen3.5 is faster at generating tokens, while Gemma3 uses less memory and responds sooner in this test. The best model depends on the task, available hardware, and answer quality.

[5]:

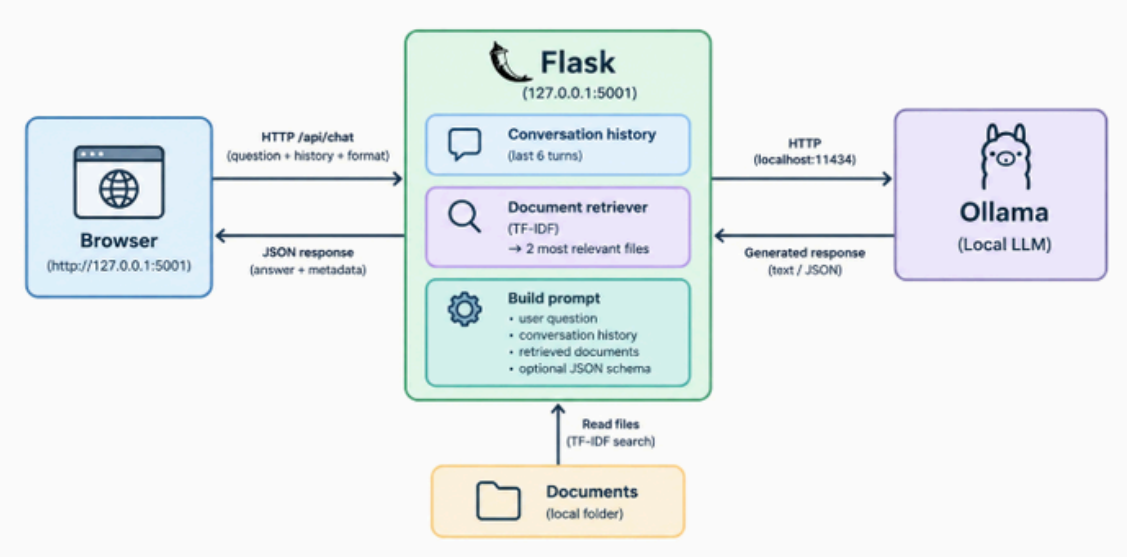
	Model	Disk size (GB)	Loaded size (GB)	VRAM (GB)	Load time (s)	Response time (s)	Tokens	Tokens/s	System RAM used (%)	Quality /5
0	qwen3.5	6.55	5.61	5.61	35.13	128.89	767	8.40	97.3	5
1	gemma3	3.35	2.91	2.91	19.35	32.81	801	72.09	93.5	4



11. FINAL PROJECT: FSTT LOCAL AI ASSISTANT

The final application is a fully local assistant that answers questions about a supplied course. It satisfies the functional requirements as follows:

• Local-only: Flask web server bound to 127.0.0.1:5001, talking to Ollama on localhost:11434; no cloud service is used.
• Knowledge from local documents: a keyword TF-IDF retriever (rag.py) selects the two most relevant files from documents/ and injects them as context.
• Conversation history: the last 6 user/assistant turns are sent with each request through /api/chat.
• Structured responses: an optional JSON Schema (topic, summary, key_points, difficulty) is passed in the format field.
• Response time: each answer displays seconds, tokens, tokens/s and the source documents used.



12. ANSWERS TO THE REPORT QUESTIONS 1/3

Q1. What is Local AI?

Local AI runs Ai models directly on my own computer instead of using a remote server

Q2. What's the difference between a Cloud LLM and a Local LLM?

Cloud LLM runs on a remote server and needs an internet connection . A local LLms runs on my own computer and can work offline

Q3. What role does Ollama play?

Ollama downloads , loads , and runs local AI models. Its also provides an API that applications can use to communicate with them

Q4. What is model quantization?

Quantization reduces the precision of model weights to decrease memory usage and often improve inference speed, with a possible small loss in quality

Q5. Why does model size affect memory consumption?

larger models contain more parameters , which require more memory to store additional memory is also needed to process inputs and generate responses

Q6. Why can a smaller model sometimes be preferable?

A smaller model uses less memory , load faster , and can run on less powerful hardware

Q7. Advantages of local inference?

local inference improves privacy , can work offline , avoid per-request cloud AP costs , and gives users more controle over their data

Q8. Disadvantages of local inference?

It requires suitable hardware , consumes local resources , and may provide lower quality or slower performance than a powerful clous model

12. ANSWERS TO THE REPORT QUESTIONS 2/3

Q9. Explain the architecture of your Python application.

the python application send a prompt to Ollama through its local API , Ollama runs the selected model and returns the generated reponse , which the application displays to the user

Q10. Explain the role of the REST API.

The REST API allows the Python application to communicate with Ollama by sending HTTP requests and receiving responses, usually in JSON format

Q11. What is RAG?

RAG (Retrieval-Augmented Generation) retrieves relevant information from external documents and provides it to the LLM to help generate more relevant and accurate answers

Q12. What's the difference between an LLM and a multimodal LLM?

An LLM processes text, while a multimodal LLM can process multiple types of data, such as text and images

Q13. Compare two local models experimentally.

Qwen3.5 uses more memory but generates tokens faster. Gemma3 is smaller and has a shorter response time in my measurements. The results show that model performance depends on memory usage, generation speed, and the task

Metric	Qwen3.5	Gemma3
Disk size (GB)	6.55	3.35
Loaded size (GB)	5.61	2.91
VRAM (GB)	5.61	2.91
Load time (s)	0.01	19.08
Response time (s)	141.86	58.82
Tokens/s	38.03	20.63
System RAM used (%)	75.7	95.1

12. ANSWERS TO THE REPORT QUESTIONS 3/3

Q14. Which model would you deploy on a computer with limited RAM? Explain.

I would choose Gemma3 because its loaded size is smaller (2.91 GB compared with 5.61 gb for Qwen3.5). However, my measurements show high system RAM usage (95.1%),so I would verify that enough memory remains available before deployment

Q15. What security risks exist when exposing a local LLM server to a network?

Ollama has no authentication by default, so unauthorized users may access the server if it is exposed. They could consume computer resources, access prompts or sensitive data, or misuse the service. Malicious documents may also contain prompt injections that manipulate the model. A firewall can restrict access , while a reverse proxy with authentication and TLS can help protect the server and encrypt communication

13. SCREENSHOTS 1/3

```
assya@ASSYA UCRT64 ~
$ ollama --version
ollama version is 0.40.11
```

01_installation.png

```
!ollama list
```

NAME	ID	SIZE	MODIFIED
qwen3.5:latest	56671c2ab938	6.6 GB	14 seconds ago
gemma3:latest	68ceab7e468a	3.4 GB	7 minutes ago

02_model.png

```
history.clear()
for q in ["My name is Ahmed.", "What is my name?"]:
    print("You:", q)
    print("AI :", ask_ai_memory(q), "\n")
```

```
You: My name is Ahmed.
AI : Nice to meet you, Ahmed! How can I help you today?

You: What is my name?
AI : Your name is Ahmed.
```

```
print("What the model actually received on the last call:\n" + "-"*50)
print("\n".join(history[:-1]) + "\nAssistant:")
```

03_chatbot.png

13. SCREENSHOTS 2/3

Architecture: `Python → HTTP POST → localhost:11434 → Ollama → Local LLM → response → Python`

Below is the same call shown step by step, including the **metadata** Ollama returns:

```
out = generate("Explain recursion in Python.")
print(out["response"][:600], "...\n")
print({k: v for k, v in out.items() if k not in ("response", "context", "thinking")})
```

```
Recursion is a fundamental programming concept where a function calls itself to solve a problem by breaking it down into smal
ntil it reaches a base case that can be directly solved. Python supports this behavior natively, though it requires careful d
cursion and stack overflows.

---

## 1. What Is Recursion?

A recursive function has two essential components:

- **Base Case:** The condition under which the function stops calling itself.
- **Recursive Case:** The logic that reduces the problem and calls the function again with smaller inputs.
 ...

{'model': 'qwen3.5', 'created_at': '2026-10-26T19:00:11.9660791Z', 'done': True, 'done_reason': 'stop', 'total_duration': 344
15555700, 'prompt_eval_count': 16, 'prompt_eval_cached_count': 12, 'prompt_eval_duration': 358089000, 'eval_count': 1294, 'ev
'wall_s': 36.462799800000084}
```

4_python.png

```
[46]: import importlib, rag
      importlib.reload(rag)

      retriever = rag.Retriever(rag.load_documents("documents"))

      def rag_answer(question, k=1, model=None):
          hits, ctx = retriever.context(question, k)
          out = generate(rag.build_prompt(question, ctx), model=model, think=False)
          return hits, out["response"], out["wall_s"]

      for q in ["What is reinforcement learning?",
                "Which hardware is needed to train neural networks with many layers?",
                "What is supervised learning?",
                "Who won the 2018 World Cup?"]:
          hits, ans, t = rag_answer(q)
          print(f"Q: {q}\n  retrieved: {hits}\n  A: {ans.strip()}\n  ({t:.1f}s)\n")
```

```
Q: What is reinforcement learning?
  retrieved: [('course.txt', 1.0831034956175303)]
  A: Reinforcement learning learns through interaction with an environment using rewards
  (3.1s)

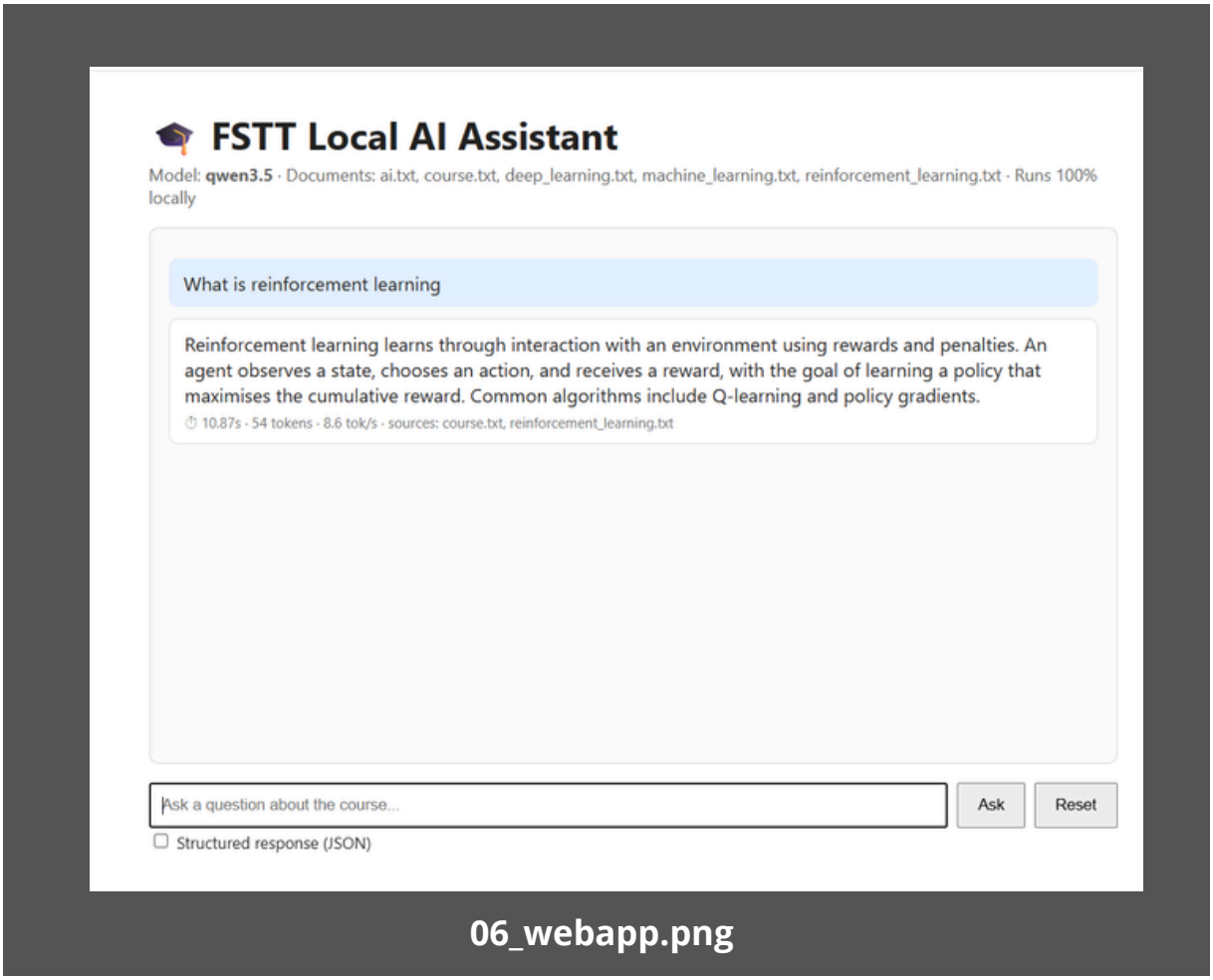
Q: Which hardware is needed to train neural networks with many layers?
  retrieved: [('deep_learning.txt', 8.42789552798877)]
  A: GPUs
  (2.8s)

Q: What is supervised learning?
  retrieved: [('course.txt', 1.0831034956175303)]
  A: Supervised learning uses labelled data.
  (3.0s)

Q: Who won the 2018 World Cup?
  retrieved: []
  A: Information not available.
  (2.7s)
```

05_rag.png

13. SCREENSHOTS 3/3



06_webapp.png